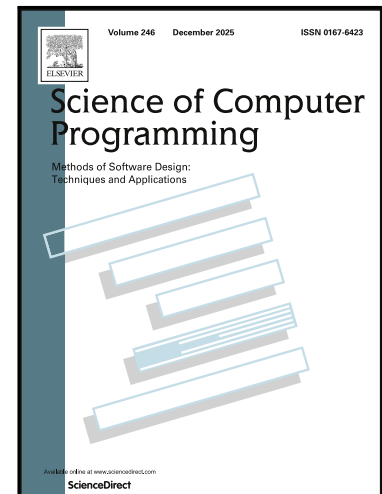


Multimodal Information Fusion for Software vulnerability detection based on both Source and Binary codes

Yuzhou Liu, Qi Wang, Shuang Jiang, Runze Wu, Hongxu Tian, Peng Zhang

PII: S0167-6423(25)00150-9
DOI: <https://doi.org/10.1016/j.scico.2025.103411>
Reference: SCICO 103411



To appear in: *Science of Computer Programming*

Received date: 30 March 2025
Revised date: 14 October 2025
Accepted date: 29 October 2025

Please cite this article as: Yuzhou Liu, Qi Wang, Shuang Jiang, Runze Wu, Hongxu Tian, Peng Zhang, Multimodal Information Fusion for Software vulnerability detection based on both Source and Binary codes, *Science of Computer Programming* (2025), doi: <https://doi.org/10.1016/j.scico.2025.103411>

This is a PDF of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability. This version will undergo additional copyediting, typesetting and review before it is published in its final form. As such, this version is no longer the Accepted Manuscript, but it is not yet the definitive Version of Record; we are providing this early version to give early visibility of the article. Please note that Elsevier's sharing policy for the Published Journal Article applies to this version, see: <https://www.elsevier.com/about/policies-and-standards/sharing#4-published-journal-article>. Please also note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

Highlights

- The study detects software vulnerability based on both source and binary codes.
- A novel fusion strategy is designed to accommodate the characteristics of two modalities.
- ChatGPT and two pre-trained models are used for the process of feature analysis process.

Journal Pre-proof

Multimodal Information Fusion for Software vulnerability detection based on both Source and Binary codes

Yuzhou Liu^{a,c,*}, Qi Wang^a, Shuang Jiang^b, Runze Wu^a, Hongxu Tian^b, Peng Zhang^a

^aCollege of Computer Science and Technology, Changchun, Jilin University, 130012, Jilin, China

^bCollege of Software, Changchun, Jilin University, 130012, Jilin, China

^cKey Laboratory of Symbolic Computation and Knowledge Engineering of Ministry of Education, Changchun, Jilin University, 130012, Jilin, China

Abstract

Context: Many researchers have proposed vulnerability detection methods to enhance software reliability by analyzing the program. However, some vulnerabilities are difficult to be identified only from the source codes, especially the ones related to the execution.

Objectives: To solve this problem, this paper introduces extra binary codes and proposes a novel solution for software vulnerability detection based on the multimodal information fusion.

Methods: The approach treats the source and binary codes as different modalities, and uses two pre-trained models as feature extractors to analyze them separately. Then, we design an attention-based information fusion strategy that taking the information from source codes as the main body while the one from binary codes as the supplement. It could not only capture the correlations among features across different modalities, but also filter the redundancy from the binary codes in the fusion process. In this way, a more comprehensive representation of software is gained and finally taken as the basis for the vulnerability detection.

Results: Our method was comprehensively evaluated on three widely-used datasets in different languages, that is Reveal in C, Devign in C++, and Code_vulnerability_java in Java: (1) For vulnerability detection performance, the Accuracy reached 86.09%, 84.58%, and 80.43% across the three datasets, with F1-scores of 82.87%, 84.62%, and 79.58% respectively; (2) Compared with seven state-of-the-art baseline methods, our approach achieved Accuracy improvements of 2.38%-3.01% and F1-score enhancements of 2.32%-8.47% across the datasets; (3) Moreover, the ablation experiment shows when combining binary codes with source codes (versus using source codes alone), the Accuracy improved by 6.83%-13.76% and F1-score increased by 5.36%-9.86%, demonstrating the significant performance gains from multimodal data integration.

Conclusion: The results show that our approach can achieve good performance for the task of software vulnerability detection. Meanwhile, ablation experiments confirm the contributions of binary codes to the detection and indicate the effectiveness of our fusion strategy. We have released the codes and datasets (<https://github.com/Wangqxn/Vul-detection>) to facilitate follow-up research.

Keywords:

software vulnerability detection, multimodal fusion, source codes feature, binary codes feature

1. Introduction

Nowadays, software products are an integral part of people's daily lives. From simple communication and entertainment to sensitive activities such as bank transfers and e-business, individuals tend to conduct everything at their fingertips. Although these software systems vary in scale and complexity, vulnerability may arise during the development process, potentially leading to serious consequences and even catastrophic impacts[1, 2] in high-risk

*Corresponding author

Email address: liuyuzhou@jlu.edu.cn (Yuzhou Liu)

systems. Consequently, software vulnerability detection has garnered remarkable attention, as it can assist developers in identifying and repairing defective modules early[3, 4], thereby improving software quality, reducing development and maintenance costs, and enhancing software reliability[5, 6].

Considering the powerful capabilities of machine learning[7], recent studies tend to approach software vulnerability detection as a binary classification problem, training deep learning models to predict whether the target data contains vulnerability. A crucial aspect of this process is creating clear and precise descriptions of the software by utilizing its various features. For instance, MVulD[8] uses the code features such as text representation, while IVDetect[9] combines features from abstract syntax trees, contextual information, and program dependency graphs. Vul-LMGNNs[10] concatenates semantic embeddings with node type embeddings, promoting cross layer propagation of code semantics and structural information through online knowledge distillation. Although these methods have yielded promising results, they primarily focus on analyzing the source codes itself.

However, some vulnerability are difficult to identify solely from the source codes[11], particularly those related to program execution, such as array out-of-bounds writing, pointer out-of-bounds access, instruction reordering, and register allocation. As shown in Figure 1, from the source codes perspective, the code segment performs buffer allocation, data writing, and read operations. Although syntactically correct and logically sound, the absence of a string terminator ‘\0’ at the buffer’s end may cause the program to read beyond valid boundaries during execution, resulting in memory access violations and unpredictable output. For example, the input “abc” of buffer may output “abc&...”. From binary codes, since binary files consist entirely of numerical information (binary digits 0 and 1) rather than human-readable text, we present their content in assembly code to make it more understandable. It can be seen that multiple MOV instructions sequentially writing characters into the buffer, but crucially lacks any instruction to write 0x00 at the buffer’s termination. This omission, though non-obvious in source codes and easily overlooked during code review, fundamentally alters memory behavior at the execution layer by permitting the processor to continue reading into unauthorized memory regions. Thus we need to modify the code by adding the statement “info->buffer[info->len]=‘\0’ ”, then the instructions “writing 0x00 at the end” would be rectified correspondingly, ensuring expected output termination.

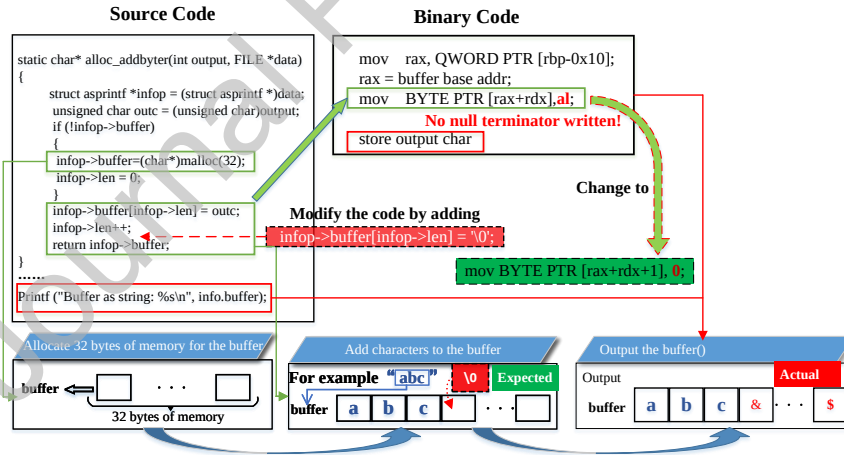


Figure 1: An example of software vulnerability

To detect software vulnerability, we must fully characterize software behavior and consider not only the source codes but also the binary codes, which contains execution information about the software. The source codes provide high-level insights, such as semantic and structural information[12], while the binary codes offer low-level details, including specific instructions. By combining the information from both sources, we can achieve a more accurate representation of the software.

In this paper, we consider the source codes and binary codes of software as different modalities, and propose a solution for software vulnerability detection through multimodal information fusion. However, several challenges arise in achieving this goal. Firstly, the subjects of software vulnerability detection are often code snippets that cannot be

compiled directly. How can we obtain their binary codes for analysis? Secondly, how can we extract useful information from different modalities, particularly from binary codes, which consist solely of strings of ‘0’ and ‘1’? Thirdly, how can we effectively fuse the information to retain valuable insights while eliminating irrelevant data[13, 14]?

Fortunately, the development of pre-trained large models can help us address these challenges to some extent:

We initially utilize ChatGPT[15] to complete code fragments, such as header files and variable declarations, when they cannot be compiled directly. By leveraging ChatGPT’s robust code completion capabilities, most code fragments can be successfully compiled after this step. Although this process may introduce some extraneous information into the binary codes, we strive to minimize the impact during the integration process.

Then, two popular pre-trained models are utilized for feature extraction across different modalities. On one hand, UniXcoder[16] is employed for source codes analysis, as it effectively captures syntactic and semantic features. On the other hand, we convert binary codes into grayscale images and use the Visual Transformer (ViT)[17] model to extract feature representations of execution information. This approach allows us to leverage the capabilities of the pre-trained models instead of training the feature extractor from scratch.

Moreover, we have developed a novel multimodal information fusion process. Clearly, compared to the information obtained from binary codes modality, the information derived from source codes modality is more accurate and exhibits less redundancy. Therefore, we prioritize source codes modality as the primary source, while using binary codes modality as a supplementary resource. We employ an attention-based fine-grained fusion method to achieve a more comprehensive representation of the software for vulnerability detection, while minimizing redundancy.

To evaluate the effectiveness of our approach, we conducted experiments using three publicly available and widely utilized datasets. The experimental results demonstrate that our method achieves Accuracy of 86.09%, 84.58%, and 80.43%, with corresponding F1-scores of 82.87%, 84.62%, and 79.58%. Compared to baseline methods, our approach yields consistent improvements, with Accuracy gains ranging from 2.38%-3.01% and F1-score improvements spanning 2.32%-8.47%, respectively, surpass those of seven other popular software vulnerability detection methods. Additionally, the ablation experiments not only highlight the value of the information obtained from different modalities but also validate the effectiveness of our information processing techniques.

In summary, our contributions in this paper are as follows:

(1) We propose a software vulnerability detection approach that integrates information from source codes and binary codes modalities, providing a more comprehensive representation of the software.

(2) Unlike previous studies, we have developed a novel information fusion process specifically designed to accommodate the features of the two modalities. This process effectively retains useful information while eliminating redundancy.

(3) We conduct extensive experiments to demonstrate the enhanced performances of our approach and analyze the effectiveness of our fusion process.

The rest of this paper is structured as follows: Section 2 systematically elaborates on binary codes acquisition methods, prompt design, feature vector formats (for both source and binary codes modalities), multimodal fusion approaches, and vulnerability detection methods. Section 3 conducts an in-depth analysis of experimental results and discusses potential limitations. Section 4 comprehensively reviews state-of-the-art software vulnerability detection techniques. Section 5 concludes the paper and outlines future research directions.

2. Methodology

The overview of our approach is shown as Figure 2, it mainly consists of three steps. ① Compiling the source codes to gain the binary codes (Section 2.1), in which ChatGPT-based code completion is given ensures code executability while minimizing additional interference with the binary generation process. ② Extract features from each modality (Section 2.2), which employs two mainstream pre-trained models for different modalities: the source codes modality is analyzed using UniXcoder to capture rich syntactic and semantic features, while the binary modality is firstly converted into grayscale images, then utilizes a Visual Transformer (ViT) to extract structural and pattern features related to program execution. ③ Feature fusion and vulnerability detection (Section 2.3). The fusion model introduces an attention-based fine-grained fusion strategy, it prioritizes retaining the source codes modality’s feature information while incorporating the binary modality as a supplementary source to enhance runtime structural information, resulting in a more comprehensive software representation. Then, a classifier is trained based on the fusion

vector for the vulnerability detection.

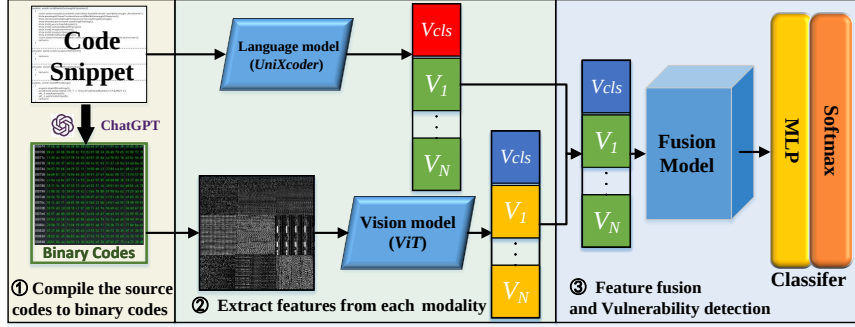


Figure 2: The overview of our approach

2.1. Gaining binary codes from source codes

Usually, the objects used in software vulnerability detection tasks are source codes[18, 19]. However, our approach aims to incorporate execution information as well. Therefore, we need to compile the source codes to obtain the binary codes.

This process could be simplified by providing the source codes directly to the corresponding compiler. However, when the object is a code snippet, the compiler may report an error due to missing necessary information. Of course, we could manually complete the missing parts if we had an object to analyze with our model in practice. But, if we want to train a model, we need an automated method to manage a large-scale dataset.

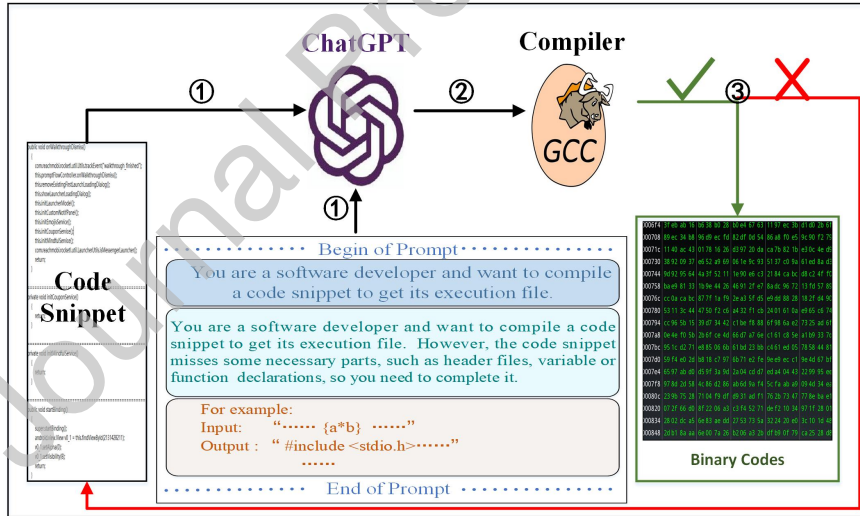


Figure 3: Code completion process

Here, we utilize the powerful ability of ChatGPT on code completion to do this job. According to our observation, the code snippet often lacks the “header files”, some “variable declarations” and “function declarations”. It is not difficult for the ChatGPT to complete these parts, so we define the task as “Completing the necessary parts of the code snippet for the compiling”.

Then, we design a comprehensive prompt for the task in the format of {Background + Task + Demonstration}. Specially:

- The Background offers an introduction to the mission scenarios, it defines ChatGPT as a software developer and gives its goal as compiling the code snippet to get its execution file.

- The Task delineates the specific objectives to be achieved: the provided code snippet misses some necessary parts such as header files, variable definitions, and function declarations, so please complete it.

- The Demonstration provides examples of the task along with the expected results. We adopt a few-shot learning approach for using LLM, that is providing 3–5 input-output pairs similar to the target task. These examples convey the required constraints and format. Compared to fine-tuning, this process does not require large amounts of labeled data and effectively guides ChatGPT to generate appropriate completions.

The process for codes completion to generate binary files is illustrated in Figure 3:

- ① The prompt and the code snippet that could not be compiled directly are given to ChatGPT.

- ② The ChatGPT completes the snippet and the results are given to a corresponding compiler.

- ③ The compiler analyzes the code: if it passes, it returns the execution file (which consists of binary codes). Otherwise, the code snippet is sent back to ChatGPT for further review.

The above process can be executed iteratively to obtain binary codes from the source codes. However, in our experiments, if a code snippet still cannot be compiled after the secondary completion, we discard it to avoid wasting resources.

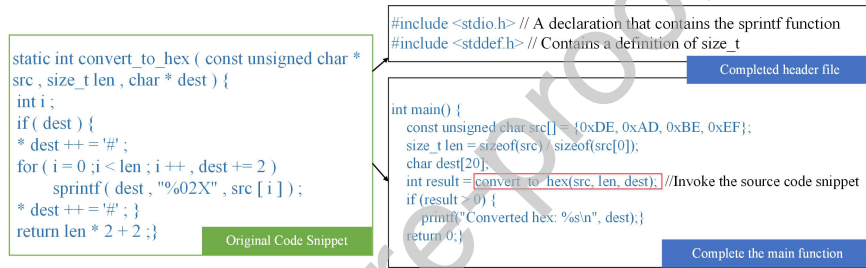


Figure 4: An example of code completion

Of course, this code completion process would introduce extra contents into the codes. Figure 4 gives an example of the completed code snippet, we can see that the completed contents are mainly the part of necessary header files and the main function for the code execution. Such information is common existed in codes, which means they does not have much impact on the vulnerability detection results (we have also conducted experiments to inspect this conclusion). Furthermore, to avoid the possible impact, our method only use the completed codes for gaining binary codes, and use the original code snippet as the materials in the subsequent analysis.

2.2. Feature representation of each modality

Feature representation refers to the process of vectorizing data in each modality separately, including Source Codes (SC) and Binary Codes (BC). In this context, we utilize two widely recognized pre-trained models to accomplish this objective.

2.2.1. Source codes feature representation

In general, source codes can be analyzed in various forms[20], such as texts and abstract syntax trees[21]. Each form has its own advantages, and some studies combine them to achieve better representations. However, this process can be time-consuming. UniXcoder is a transformer-based programming language model that can help us obtain similar representations easily, as it has been pre-trained on large-scale datasets. Therefore, we use UniXcoder[16] as the feature extractor for source code.

In addition, to enhance the subsequent information fusion process, we modify the code sequences and add three additional types of tokens:

•<CLS>, which aims to capture the semantic information of the entire sequence, providing a global representation of the input.

•<Encoder only>, which is used to specify the working mode of the model, indicating that we are performing encoding tasks without involving decoding or generation processes.

•<SEP>, which serves as a separator for dividing source codes and tags, helping the model understand the different parts of the input sequence.

By incorporating these tokens, the input can be tokenized as follows:

$$T = \{<CLS>, <Encoder - only>, <SEP>, Source\ code, <SEP>\}.$$

Then, UniXcoder analyzes the above token sequence and the output is obtained as $T_{sc} = \{<CLS>, t_1, t_2, \dots, t_n\}$, where the vector <CLS> represents the entire information of source codes, we donate it as V_{sc} and take it as the basis for the subsequent process.

2.2.2. Binary codes feature representation

Since binary codes are challenging to analyze directly, our study is inspired by prior researches in the field of malware detection, where binary codes are visualized as grayscale images for pattern recognition. Pinhero et al.[22] has shown that malware and benign software present distinct visual patterns when represented as grayscale images. Specifically, Nataraj et al.[23] has shown that malware images often exhibit repetitive or locally dense texture structures, while benign samples tend to have smoother and more uniform texture distributions. These differences imply that binary data, when mapped into image space, may carry discriminative texture or structural patterns. Motivated by these insights, we complete function fragments and compile them into binary files, which are then converted into images to explore potential vulnerability-related visual patterns.

In our experiments, we also observed that certain classes of vulnerable samples show localized grayscale intensity variations or block-like structures in their image representation, while non-vulnerable samples tend to appear more uniformly distributed. These differences are also captured by the attention regions of the ViT model.

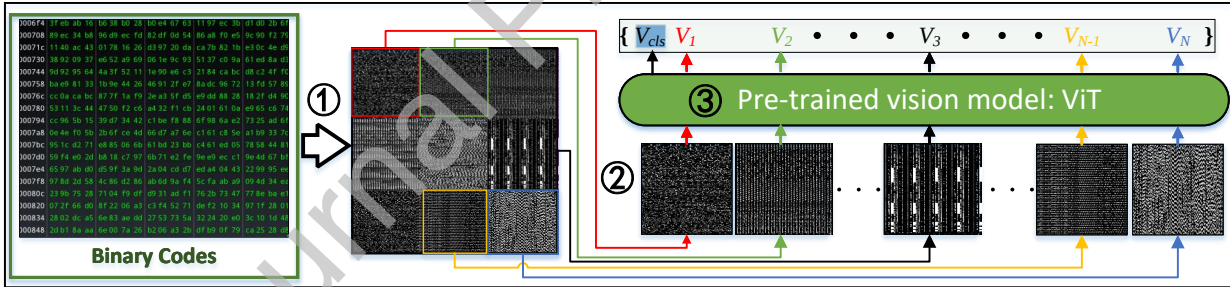


Figure 5: Binary code extraction process

To support this observation, we conducted image feature analyses using three quantitative metrics:

•LBP energy (for local texture patterns): Radius = 3 px, 8 sampling points, uniform mode (low-transition patterns only), histogram energy, normalized with a $1e-6$ smoothing factor.

•FFT peak intensity (for frequency-domain features): 2D FFT on grayscale images, zero-frequency centered, maximum peak intensity from the central 20×20 spectrum region.

•Structural Entropy (for edge coherence): Sobel edges, 32-bin local histograms in 7×7 windows, entropy $\times$ edge strength, min-max normalized.

In statistical analysis, grayscale image samples derived from binary files of vulnerable and normal groups, and independent-sample t-tests were performed on each feature type. Bonferroni correction (three tests) was applied to control false positives, and significance was set at p less 0.0001. Statistical significance tests show that for all three metrics, the p -value was less 0.0001, confirming a significant difference in image texture between the two classes. Meanwhile, an examination of the statistical data for the three metrics reveals that:

① Vulnerable samples had higher LBP energy, suggesting more complex local textures.

② In the frequency domain, their images had distinct peaks, while benign samples had smoother, more evenly distributed energy.

③ Structural entropy was significantly higher in vulnerable samples, likely due to code obfuscation, encryption, or padding, which disrupt edge continuity.

This empirical evidence supports the viability of using image-based representation for capturing semantic and structural signals related to vulnerability.

Based on above analysis, we convert binary codes into grayscale images and extract visual patterns as BC features using the pre-trained model ViT[17]. There are many well-established tools and algorithms in the field of image processing, which make it easy to perform further analysis of the generated images. This process consists of three main steps, as illustrated in Figure 5.

Firstly, the binary codes are converted into a grayscale image. Since the eight binary byte sequences correspond to a certain number within the $[0, 255]$, we map every eight characters to the pixel values of the image. Suppose that there are total n bytes for the binary codes, the number of pixels we gained is $P = N/8$. Then, these pixels are reshaped into a square, where the length of the side is $L = \lceil \sqrt{P} \rceil$. Here, $\lceil \sqrt{P} \rceil$ means $\sqrt{P}$ rounding upwards. For the final empty part of the image, we use a simple zero filling strategy.

Secondly, the image is clipped into small blocks to cope with the model requirement. To simplify the model structure, implementation, and improve computational efficiency, the input of ViT needs to be square and have same batch size. Thus, for the given image $X \in R^{H \times W}$, where H and W are the height and width of the image, image X is clipped into N fixed size image blocks $Z_i, i \in [1, N]$. Referring to existing research, we have uniformly determined the grayscale image size to be 256, that is each block is a 16×16 image.

Finally, each image is further handled by ViT to get its corresponding vector representation. Meanwhile, the Position encoding and [CLS] labeling are introduced during linear transformation. Position encoding ensures that the network can distinguish image blocks at different positions, while [CLS] labeling aggregates the features of the entire input image.

In this way, we get the representations of binary codes as a set of vectors, that is $V_{BC} = \{V_{CLS}, V_1, V_2, \dots, V_N\}$, where summarizes the entire information and represents the information of each image block.

2.3. Feature fusion and Vulnerability detection

This is the central aspect of our approach. It integrates information from source codes and binary codes modalities to create a comprehensive representation of the software, which serves as the foundation for generating vulnerability detection results.

2.3.1. Architecture of the module

Whether using direct concatenation or fusion model methods, prior studies typically treat each modality equally in the fusion process[24]. However, due to differences in analysis accuracy, the information obtained from source codes and binary codes varies in quality, specifically, source codes are purer. Based on this characteristic, we have designed a novel fusion strategy. The architecture of our feature fusion and vulnerability detection modules is illustrated in Figure 6.

The initial input of this process consists of features extracted from source codes V_{SC} and a set of feature vectors derived from binary codes $V_{BC} = \{V_{CLS}, V_1, V_2, \dots, V_N\}$. The overall concept of our fusion strategy is to use V_{SC} as the foundation and integrate valuable supplemental information from V_{BC} to get the final comprehensive representation. Thus, we firstly remove the V_{CLS} from V_{BC} , as it lacks sufficient detail, then, V_{SC} is added to form the actual input, that is $\{V_{SC}, V_1, \dots, V_N\}$, so that the model can learn both the features of the source codes and the visual characteristics of the image in subsequent operations, further enhancing its cross-modal understanding ability.

Furthermore, our fusion module consists of three main steps, which we will illustrate in detail as follows:

(1). Attention calculation

This step employs a multi-layer attention mechanism[25] that automatically identifies the positions of the most important information. It then dynamically adjusts the representation of each position and ultimately captures the interdependence between positions to enhance the richness and flexibility of the representation. To achieve this goal, we stack n transformer blocks, and the specific calculation process is given next.

Suppose that the dimension of the input is $(N + 1) \times D$, where N is the number of patches in the image, plus one

represents the V_{SC} token, and D is the dimension of each token. For each layer of input, we obtain the query Q , key K , and value V through linear transformation. The dimensions of these matrices are $(N + 1) \times D$.

$$Q = XW_Q, K = XW_K, V = XW_V \quad (1)$$

Where, W_Q, W_K, W_V are the parameter matrices. Then, attention weight is calculated by dot product of query Q and key K . Next, the representation of each token is updated by multiplying the attention weight matrix with the value matrix V .

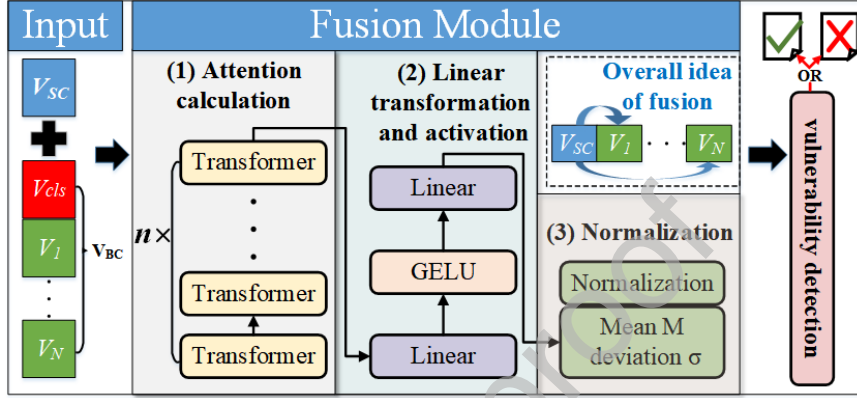


Figure 6: Overall architecture of our approach

For the V_{SC} Token, its query vector is calculated by taking the dot product with the key vectors of all other tokens to determine its correlation with them and generate the corresponding attention weights. The value vector of the V_{SC} token will be weighted and summed with all value vectors (including the V_{SC} token itself) using the attention score matrix to obtain the updated representation.

Finally, the output from each layer is passed as input to the next layer until all layers have been processed. In this manner, the final V_{SC} represents the global feature representation of the fused vector, which we utilize as input for subsequent operations.

(2). Linear transformation and activation

To explore complex patterns, we perform linear transformation and activation operations that enable the output vector to capture more feature information and enhance its expressive power. This process consists of three components: “linear transformation $\rightarrow$ activation function $\rightarrow$ linear transformation”, as illustrated in Figure 6. Firstly, the vector undergoes a linear transformation to increase its dimensionality. Next, a nonlinear activation function, specifically the Gaussian Error Linear Unit (GELU), is applied to introduce nonlinearity, allowing the model to learn more intricate mapping relationships. Finally, the output vector is subjected to another linear transformation to restore its dimensionality, yielding the final result. The dimension is reverted to its original state to serve as input for normalization processing, thereby facilitating subsequent calculations and operations.

(3). Normalization operation

The final step is routine normalization, which can enhance the stability and convergence of the model while maintaining a consistent data distribution. This process helps reduce the likelihood of gradient vanishing or exploding issues. Normalization is achieved by calculating the mean μ and standard deviation σ . By subtracting the mean and dividing by the standard deviation, the transformed vector’s mean is adjusted to 0, and its standard deviation is adjusted to 1. This ensures that the optimizer does not update the weights with excessively large or small gradients due to scale differences in the input features, thereby improving training efficiency and serving as the input for the final prediction phase.

After the fusion module, the information from the source codes modality and binary codes modality is integrated to obtain the final representation vector, which is then input into a classifier for software vulnerability detection. Any type of classification can be employed. However, we have chosen a straightforward approach (Multilayer perceptron (MLP) with Softmax activation) to better demonstrate the representational capability of our fusion vector.

2.3.2. Model training

Software vulnerability detection can be treated as a binary classification problem. Therefore, we use cross-entropy[26] loss as our model training objective. During training, the parameters of the source codes modal and execution codes modal feature extractors remain unchanged because they are built on a large-scale pre-trained model. On the one hand, due to the difficulty of collecting labeled data and the relatively limited amount of data, direct fine-tuning may lead to overfitting problems on specific datasets. On the other hand, inappropriate fine-tuning can also undermine the model's original analytical capabilities. As a result, we only update the feature fusion and classification modules, which contain a smaller number of parameters and are relatively less expensive to train. In this way, the multi-head self-attention mechanism in the feature fusion network can effectively capture the feature relationship between modalities, and the optimization process based on cross-entropy loss integrates the task-related knowledge into the vulnerability detection, so as to improve the applicability of the fusion feature in the task. This loss function quantifies the difference between the predicted label generated by the vulnerability predictor and the ground-truth label. The formula for calculating cross-entropy loss is shown below.

$$Loss = - \sum_{i=1}^n \sum_{c=1}^2 y_{i,c} \log(p_{i,c}) \quad (2)$$

Where $y_{i,c}$ is the true label, $p_{i,c}$ is the predicted probability, n represents the number of test samples. According to this loss, we optimize the feature fusion module and vulnerability predictor as a cohesive unit, allowing our fusion process to more effectively capture domain-specific knowledge in software vulnerability detection. We employ a supervised strategy for training the model using labeled data. The detailed training process is described in [Algorithm 1](#).

Algorithm 1: Fusion and Prediction Training

Input: Source code features ,execution code features and labels in the dataset

Output: The predicted labels

```

1 Initialize  $W_{Fusion}, W_{Classify}$ 
2  $D_{train}, D_{test} = \text{split}(\text{input})$ 
3 # Training Phase
4 for epoch = 1 to max_epoch do
5    $V_{Fusion} = F^{Fusion}(D_{train})$ 
6    $y \leftarrow F^{Classify}(V_{Fusion})$ 
7    $Loss = - \sum_{i=1}^n \sum_{c=1}^2 y_{i,c} \log(p_{i,c})$ 
8   Optimizer Loss
9   Update Parameters  $W_{Fusion}, W_{Classify}$ 
10 end
11 Evaluation in  $D_{test}$ 

```

In lines 1-2, the input consists of the extracted source codes and binary codes feature vectors. We randomly initialize the parameters (W^{Fusion} and $W^{Classify}$) involved in the fusion process (F^{Fusion}) and the classification process ($F^{Classify}$).

During training phase (lines 4-6), the fusion vectors are derived from the training set to make classification predictions. Subsequently, the loss is calculated based on the predicted probabilities and the actual labels.

We optimize lines 7-9 by employing backpropagation to calculate gradients, which are used to update the parameters involved in the training and fusion process. This approach minimizes the loss function as much as possible, leading to optimal results.

When extracting features from source codes and binary codes, we utilize pre-trained large models that typically contain a considerable number of parameters. Backpropagation and gradient calculations demand substantial memory and computational resources. If every parameter is involved in gradient updates, the training process can become exceedingly slow. Additionally, if all parameters are optimized during training, the model may inadvertently memorize noise and specific details from the training data, resulting in a loss of generalization ability, which hinders the improvement of final prediction performance. Therefore, we selectively optimize only the parameters related to feature

fusion and prediction.

3. Experiments demonstration

To evaluate the effectiveness of our method, we conducted a series of experiments and examined the following four questions from different perspectives:

RQ1(Performance evaluation): How does our method perform in predicting software vulnerabilities?

RQ2(Completion impact): How does the code completion process impact the vulnerability detection results?

RQ3(Ablation experiment): Whether each part of our method (especially the fusion method) is useful?

RQ4(Parameter analysis): What are the correct model parameter settings for our method?

The primary objective of RQ1 is to assess the overall performance of our method by comparing it with seven widely recognized software vulnerability detection techniques. In RQ-2, we verified whether code completed by Chat-GPT would have an impact on prediction performance. In RQ3, we examined the utility of the three key components of our approach: source codes modality, binary codes modality, and the fusion method. Furthermore, since fusion is the central element, we conducted a more in-depth analysis by comparing different fusion methods to evaluate the effectiveness of each step. In relation to RQ4, we investigated the parameter settings of the model.

3.1. Implementation and Experiment Setup

1)Experimental environment: Based on python 3.10, we also implemented the model function with the help of Transformer and MLP, and all experiments were conducted with the following experimental configurations: 10th generation Intel(R) Core(TM) i7-10700 2.90GHz with 16GB memory; NVIDIA GeForce RTX 2080 Ti, 22GB RAM; 64-bit Windows 11.

2)Dataset: To fully evaluate the performance of our method in vulnerability detection, and considering that a single-language dataset cannot capture the full range of vulnerability manifestations, this study selects datasets from three different programming languages to ensure a comprehensive assessment. We present the dataset characteristics in Table I, detailing the number of vulnerable/non-vulnerable samples, data sources, and language types.

Table 1: Dataset Statistics Comparison

Dataset	#Vulnerable	#Non-Vul	Source	Type
ReVeal	1,664	16,505	Chrome, Debian	C
Devign	10,035	11,358	Github	C++
code_vulnerability_java	527	449	Hugging Face	Java

- REVEAL[27] offers paired C source code samples in both vulnerable and patched states, systematically constructed by analyzing historical vulnerabilities from two major open-source projects: the Linux Debian Kernel and Chromium.

- The Devign[28] dataset was constructed from four large-scale open-source projects (Linux Kernel, QEMU, Wireshark, and FFmpeg), where the creators collected and manually labeled function-level vulnerability data. Notably, finer-grained annotations like vulnerable line numbers were unavailable in its initial release.

- The code_vulnerability_java[29] dataset was created through systematic code analysis and vulnerability detection techniques, with its developers extracting representative snippets from real-world projects to ensure data diversity and practical relevance.

3)Evaluation metrics: In our experiment, we applied four commonly accepted metrics in the field of software vulnerability detection for evaluation.

$$Accuracy = \frac{TP + TN}{TP + FN + FP + TN} \quad (3)$$

$$Precision = \frac{TP}{TP + FP} \quad (4)$$

$$Recall = \frac{TP}{TP + FN} \quad (5)$$

$$F1 - score = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (6)$$

Among them:

True Positive (TP): TP indicates that the method predicts a vulnerability software sample as a vulnerability software.

False Positive (FP): FP indicates that a benign sample is predicted as a vulnerability sample by the method.

False Negative (FN): FN indicates that the method predicts a vulnerability software sample as benign.

True Negative (TN): TN indicates that the method predicts a benign sample as benign.

3.2. Experiments for RQ1

The purpose of RQ1 is to evaluate the performance of our proposed method for vulnerability detection and to compare it with other baseline models. In our model configuration, the learning rate and batch size are set to $1e-3$ and 32, respectively, with training conducted for 10 epochs. The dropout rate is set to 0.1, and the optimizer is uniformly Adam. To ensure fairness, all models were trained under identical experimental conditions. For models without publicly available official codes (e.g., Acos), we rigorously reproduced its methods following the original paper descriptions, while the remaining models used their open-source implementations to maintain consistency. The dataset was divided into a training set and a testing set, with a ratio of 8:2.

To assess the performance of our method, we selected seven widely used software vulnerability detection models as baselines for comparison. To make the comparison more comprehensively, the baselines use the same materials, including source codes, binary codes, or both of them. Detailed information is provided in Table II.

Table II: The compared methods

Name	Published Paper	Materials	Date
DTL-DP	Software Visualization and Deep Transfer Learning for Effective Software Defect Prediction	Source code	2020
IVDetect	Vulnerability Detection with Fine-Grained Interpretations		2021
AMPLE	Vulnerability Detection with Graph Simplification and Enhanced Graph Representation Learning		2023
Acos	A cosine similarity-based labeling technique for vulnerability type detection Using source codes		2024
VulHawk	VulHawk: Cross-architecture Vulnerability Detection with Entropy-based Binary Code Search	Binary code	2023
Vul-Transformer	Vulnerability detection through cross-modal feature enhancement and fusion	Both	2023
Vul-CoCA	Transformer-based statement level vulnerability detection by cross-modal fine-grained features capture		2025

- DTL-DP[30], which is a deep learning framework that utilizes code images and attention mechanisms.

- IVDetect[9], which detects vulnerabilities through graph convolutional network and a feature attention mechanism.

- AMPLE[31], which combines graph simplification and enhanced graph representation learning to improve the performance of vulnerability detection.

- Acos[32], which is a vulnerability detection model based on cosine similarity. Please note, since the author did not provide a name for the model, we refer to it as Acos.

- VulHawk[33], which uses the Intermediate Representation Function Model, entropy-based adapters, and progressive search strategies to enable cross-architecture binary codes vulnerability detection.

- Vul-CoCA[34], which enhances software vulnerability detection by integrating cross-modal feature enhancement and fusion of source codes and binary codes. The method aligns statements with instructions and variables with

registers, then applies co-attention mechanisms to capture semantic correlations between the two modalities.

•Vul-Transformer[35], which enhances statement-level vulnerability detection by integrating cross-modal fine-grained feature alignment between source code and assembly code with Transformer-based architecture. The method aligns statements with instructions and variables with registers, then applies bimodal slice purification to remove noise.

We evaluated our proposed method and baseline models on three benchmark datasets: Reveal, Devign, and Code_vulnerability_java. The comparative results are summarized in Table III. Our approach consistently outperforms all baselines across all datasets. On Reveal: 86.09% Accuracy (+2.38% over best baseline) and 82.87% F1-score (+8.47%); On Devign: 84.58% Accuracy (+2.38%) and 84.62% F1-score (+5.92%); On code_vulnerability_java: 80.43% Accuracy (+3.01%) and 79.58% F1-score (+2.32%).

Through the analysis of the data in Table III, it can be found that our method has achieved the best results in the evaluation indicators, compared with other models, and this comprehensive performance improvement fully proves the effectiveness of our proposed innovative method in feature extraction and vulnerability identification. And there are three potential reasons for this.

Table III: Comparison with baselines(A:Accuracy; P: Precision; R: Recall; F: F1-score)

	Reveal				Devign				Code_vulnerability_java			
	A	P	R	F	A	P	R	F	A	P	R	F
DTL-DP	0.5743	0.6349	0.7562	0.6903	0.6510	0.7353	0.8078	0.7353	0.6987	0.7237	0.6987	0.6924
IVDetect	0.3736	0.3447	0.5163	0.4134	0.6544	0.6250	0.7251	0.6713	0.6472	0.7725	0.5000	0.6071
AMPLE	0.8371	0.4706	0.4057	0.4357	0.5716	0.5064	0.7899	0.6194	0.5518	0.7600	0.5518	0.4529
Acoss	0.7171	0.7737	0.7498	0.7615	0.6360	0.6276	0.6691	0.6477	0.6456	0.6341	0.6882	0.6601
VulHawk	0.7016	0.6496	0.7564	0.6989	0.5858	0.6294	0.5765	0.5335	0.6695	0.7179	0.6622	0.6439
Vul-Transformer	0.7522	0.7439	0.7380	0.7410	0.8020	0.8240	0.7510	0.7870	0.7725	0.7752	0.7600	0.7675
Vul-CoCA	0.7124	0.7143	0.7764	0.7440	0.8220	0.7490	0.7620	0.7550	0.7742	0.8019	0.7454	0.7726
Ours	0.8609	0.8251	0.8324	0.8287	0.8458	0.8180	0.8763	0.8462	0.8043	0.8321	0.7625	0.7958

1) By utilizing these two complementary sources of information, we enhance our understanding of vulnerability information and improve the model's ability to identify more complex issues.

2) We use pre-trained models as feature extractors for each modality of data. Since these models are trained on large-scale datasets, they can provide us with superior features for information representation. This is beneficial for subsequent information fusion, allowing for improved descriptions of the software.

3) Compared to traditional manual feature design methods, our cross-modal attention mechanisms can automatically learn and identify relationships between different modalities. This enhancement also improves the model's flexibility and generalization capabilities.

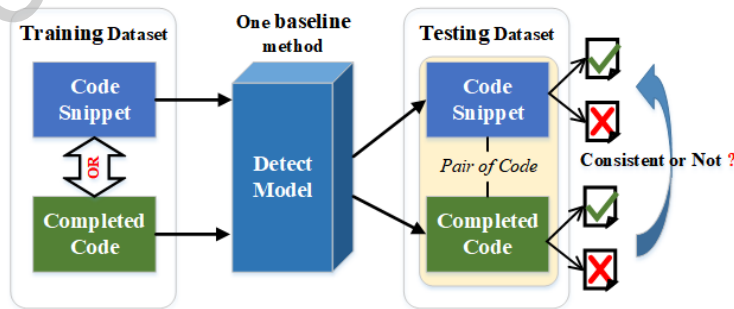


Figure 7: Experiment design for RQ2

3.3. Experiments for RQ2

In fact, it is difficult to directly verify whether the completed codes aligns with the original one for the vulnerability detection task. Thus, we design the experiments to inspect this question indirectly based on the idea: if the model gives the consistent prediction results on a code snippet and its completed one, we believe they align with each other as our completion process has little impact on the prediction. Figure 7 shows the data processing process of RQ2.

Table IV: Consistency Rate of Different Methods Across Groups (%)

Methods	Groups					
	Reveal-Def	Reveal-Non	Devign-Def	Devign-Non	CodeVul-Def	CodeVul-Non
AMPLE	99.83	99.84	99.54	99.91	99.51	99.79
IVDetect	99.72	99.74	99.72	99.85	99.58	99.82
DTL-DP	99.56	99.66	99.68	99.59	99.63	99.86
Vul-Transformer	99.68	99.73	99.61	99.72	99.67	99.89
Vul-CoCA	99.73	99.85	99.47	99.88	99.72	99.93
Acos	99.61	99.56	99.64	99.72	99.76	99.97

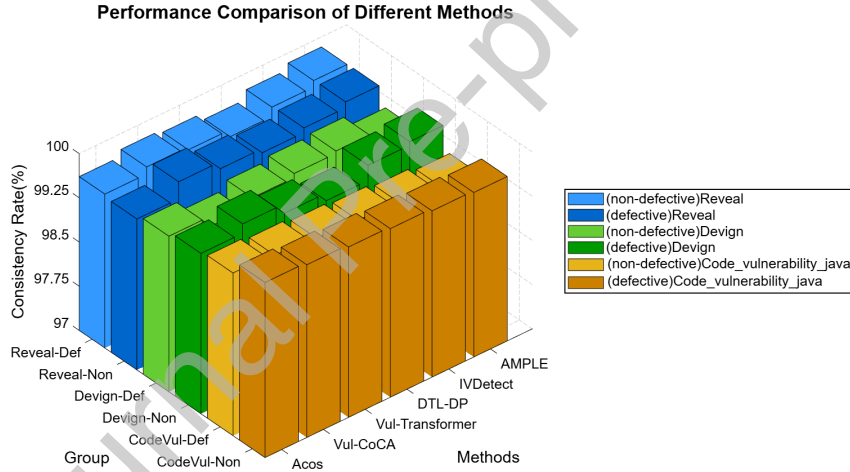


Figure 8: Consistency rate of different baselines

In the experiment, we establish the dataset with pairs of codes $\langle C, C' \rangle$, where C is the original code snippet and C' is its completed code with our method. To ensure the fairness and consistency of the experiments, we exclusively selected code segments that were completed by ChatGPT and could be compiled successfully, along with their corresponding completed files, as the final dataset. This unified dataset was used for training and testing across all methods, thereby eliminating potential biases introduced by varying input format requirements among different models. Then, the dataset was divided into training dataset and testing one by the ratio 8:2. In each round of experiment, we randomly select C or C' from each pair in the training dataset to train the detection model with some baseline methods, as shown in Table II.

Firstly, it is important to emphasize that: since extracting binary codes corresponding to code snippets before completion is highly challenging if they can not be complied directly, we exclusively used source codes as input for the methods mentioned in this section. However, VulHawk only accepts binary codes so it could not participate in this validation experiment. For baseline methods requiring binary codes inputs (e.g., Vul-CoCA and Vul-Transformer), since their full implementation depends on specific compilation toolchains and preprocessing pipelines, we evaluated only their source codes feature extraction modules in our experiments. This approach ensures that all models are

compared under consistent input conditions.

Furthermore, since our method requires the fusion of binary information, we leveraged comparative results from other baselines to indirectly demonstrate the rationality of our approach. The experiment corresponding to Figure 7 was designed as a supplementary analysis to explore the impact of code completion on prediction consistency across different models. Specifically, we analyzed whether the predictions of multiple baseline models changed before and after code completion, thereby assessing their sensitivity to completion-induced modifications.

Finally, each pair of codes in testing dataset are analyzed by the prediction model to judge whether it is vulnerable. We define the consistency rate as the percent of completed codes that having same prediction label with the original codes. For example, there are 10 original codes were predicted as vulnerable by one model but only 8 completed codes of them have the same label, the consistency rate is 80%. Totally, 10 rounds experiments were conducted, and the average of them are taken as the final results.

Figure 8 shows the performance comparison in terms of consistency rate across the three datasets, and Table IV is additionally provided to present the specific values. By balancing visual effects and data clarity, we can observe that: whether for the vulnerable or non-vulnerable codes, more than 99.5% pairs of codes before and after completion have the consistent prediction labels with different baseline methods. Meanwhile, for the inconsistent pairs, most of their prediction probabilities tend to be intermediate, which means the model may give different prediction results on these codes themselves, and the inconsistency in the experiment is caused by the randomness of the model. Thus, the experimental results show that the performance of these baseline methods exhibits only minor variations after code completion, suggesting that the impact on our method should also be relatively limited. Moreover, given that our method shares structural and encoding similarities with these baselines (e.g., using token sequencebased input representations and Transformer-based architectures to model code context), this inference is logically reasonable. Due to this consistency in input modeling and encoding processes, we have grounds to believe that the input perturbations introduced by code completion would not significantly deviate from the trends observed in baseline methods.

3.4. Experiments for RQ3

Multimodal fusion is the central concept of our approach, so we further investigate how it impacts the performance of vulnerability detection. Therefore, we focus on two important aspects in this study: 1) whether the information from the two modalities is significant. And 2) whether our feature fusion process is logical.

3.4.1. Modality ablation

In the experiments, we utilized the pre-trained models UniXcoder and ViT as feature extractors to analyze either source codes or binary codes, while keeping all other experimental conditions constant.

The experimental results are presented in Table V. The best performance is achieved when utilizing both modalities, indicating that both are essential for our approach.

Table V: Performance of each modality

Modality	Dataset	Accuracy	Precision	Recall	F1-score
ViT+binary	Reveal	0.4559	0.4622	0.5392	0.4977
	Devign	0.6140	0.6766	0.4439	0.5361
	Code_vulnerability_java	0.6992	0.6534	0.5711	0.6095
UniXcoder+source	Reveal	0.7926	0.8127	0.6826	0.7420
	Devign	0.7120	0.7179	0.7088	0.7133
	Code_vulnerability_java	0.7325	0.6518	0.6716	0.6616
Both of the two modalities	Reveal	0.8609	0.8251	0.8324	0.8287
	Devign	0.8458	0.8180	0.8763	0.8462
	Code_vulnerability_java	0.8043	0.8321	0.7625	0.7958

One more step reveals that the performance of the ViT alone is quite poor, indicating that it extracts little to no useful information from binary codes. In contrast, the performance of UniXcoder is better, but still not satisfactory.

After integrating the information from both modalities, the performance reaches its peak. This suggests that: 1) most of the irrelevant information obtained from binary codes has been eliminated; and 2) supplementary information has been effectively combined with the data from source codes, enhancing the model's ability to predict vulnerability.

3.4.2. Evaluation of our feature fusion process

Although the previous findings have indirectly verified the effectiveness of our fusion method, we still wish to examine it more thoroughly from two specific aspects: ①How does our method perform in comparison to other common fusion methods? ②Is each step in our fusion process beneficial?

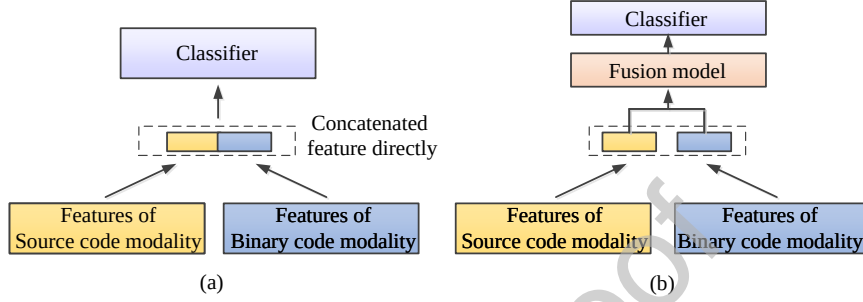


Figure 9: The compared fusion methods

①For the first sub-aspect, we selected two common fusion methods for comparison, as illustrated in Figure 9: (1) the most prevalent multimodal fusion method involves directly concatenating feature vectors from different sources, as illustrated in Figure 9(a); and (2) an alternative approach where an additional fusion model is trained on top, as depicted in Figure 9(b). Furthermore, the fusion model in Figure 9(b) is important, and we have chosen four classical models for comparison: RNN[36], LSTM[37], GRU[38], and GCN[39].

In the experiment, only the fusion component of our approach was modified, while ensuring that the experimental conditions remained consistent. The comparison results are presented in Table VI. We can draw the following observations:

Table VI: Compare with other fusion methods

		Dataset	Accuracy	Precision	Recall	F1-score
(a)		Reveal	0.7452	0.7928	0.7034	0.7454
		Devign	0.6933	0.6625	0.7853	0.7187
		Code_vulnerability_java	0.7251	0.6934	0.7028	0.6981
(b)	RNN	Reveal	0.7804	0.7648	0.7719	0.7683
		Devign	0.7809	0.8057	0.7419	0.7725
		Code_vulnerability_java	0.7040	0.6839	0.7803	0.7289
	LSTM	Reveal	0.7828	0.7800	0.7749	0.7774
		Devign	0.8118	0.7660	0.7390	0.7523
		Code_vulnerability_java	0.6890	0.6705	0.7672	0.7156
	GRU	Reveal	0.7853	0.8129	0.7759	0.7940
		Devign	0.7721	0.7981	0.7302	0.7626
		Code_vulnerability_java	0.7140	0.6903	0.7467	0.7174
	GCN	Reveal	0.8021	0.7934	0.7948	0.7941
		Devign	0.8029	0.8071	0.7977	0.8024
		Code_vulnerability_java	0.7577	0.8366	0.6737	0.7464

Firstly, when employing the fusion method (a), the performance is the lowest, comparable to using only the information from the source codes modality, as shown in Table V (F1-score: 74.54% vs. 74.20% in Reveal). This

indicates that the information from the binary codes modality is not being utilized effectively for prediction, and direct concatenation fails to capture the correlation between different modal features in our task.

Secondly, compared to direct concatenation, all models utilizing the fusion method (b) demonstrate improved performance. Notably, in Devign dataset, when employing the GCN as the fusion model, the F1-score reaches 80.24%, reflecting an approximate 8% enhancement. This suggests that the additional fusion model effectively integrates information from the two modalities.

Thirdly, none of these methods can achieve performance as good as our approach. In Reveal dataset, even the best method, “(b)+GCN”, shows a degradation of 3.46% (79.41% vs. 82.87%). This demonstrates that our fusion process outperforms these methods, primarily because it fully utilizes the main body information from source codes while incorporating supplementary information from binary codes, allowing for the filtering of most redundancies.

② For the second sub-aspect, our fusion method primarily consists of three components, as introduced in Section 2.1) Attention calculation, 2) Linear transformation and activation and 3) Normalization. To assess the utility of each step, we compared different combinations of these components to analyze their performance.

The results are presented in Table VII. The following observations can be made: 1) The effect of attention calculation is notable and performance improves significantly when it is applied. 2) When attention calculation is utilized, the performance of linear transformation and activation is slightly better (by approximately 1%) compared to the normalization operation. 3) When only linear transformation and activation, and normalization are used, the performance is the lowest and comparable to the direct concatenation fusion method shown in Table VI (F1-score: 75.26% vs. 74.54% in Reveal dataset). 4) Utilizing all the steps yields the best performance.

In response to the observation mentioned above, we have conducted the following analysis:

1) By employing multiple Transformer layers, the model can learn more nuanced feature representations. The self-attention mechanism of the Transformer effectively captures the dependency relationships between different components and facilitates better gradient transfer, thereby mitigating issues related to gradient vanishing or exploding. This capability is particularly important for analyzing software vulnerability.

2) Understanding the contextual relationships and long-term dependencies of input data are essential when addressing complex inputs. Simple feature transformations or shallow networks are inadequate for fully capturing these complex relationships.

In summary, each step of our proposed fine-grained fusion process is valuable. While each step has varying levels of importance, they collectively contribute to enhancing overall performance.

Table VII: Rationality analysis of fusion strategy across datasets

Attention	Linear	Norm.	Dataset	Accuracy	Precision	Recall	F1-score
✓	✓	×	Reveal	0.8473	0.8204	0.8043	0.8122
			Devign	0.7733	0.7657	0.8377	0.8001
			Code_vulnerability_java	0.7059	0.7780	0.7569	0.7673
✓	×	✓	Reveal	0.8369	0.8079	0.8014	0.8046
			Devign	0.8039	0.7724	0.8631	0.8152
			Code_vulnerability_java	0.8013	0.8089	0.7437	0.7749
×	✓	✓	Reveal	0.7452	0.7928	0.7164	0.7526
			Devign	0.7550	0.7526	0.7262	0.7392
			Code_vulnerability_java	0.7057	0.7063	0.7338	0.7197
✓	✓	✓	Reveal	0.8609	0.8251	0.8324	0.8287
			Devign	0.8458	0.8180	0.8763	0.8462
			Code_vulnerability_java	0.8043	0.8321	0.7625	0.7958

3.5. Experiment for RQ4

According to RQ3, we identify that the Transformer architecture plays a crucial role in vector processing following fusion. Therefore, we investigate the impact of the number of attention heads (num_heads) and network layers

(num_layers) on final prediction performance to determine the optimal parameters configuration.

Num_heads and num_layers are the core structural parameters of the Transformer architecture in our study, so they directly determine the model’s multimodal feature fusion capability for binary and source code. Specifically: 1) The Num_heads fundamentally affects the granularity of cross-modal feature interaction in the multi-head attention mechanism. 2) The num_layers governs the hierarchy of feature abstraction.

As for other hyperparameters such as learning rate and batch size, these parameters mainly influence training stability and convergence speed, their variations primarily affect optimization dynamics rather than the representational capacity of the model architecture itself. Thus, we adopted widely-used standard settings in our experiments and performed preliminary tuning on the validation set.

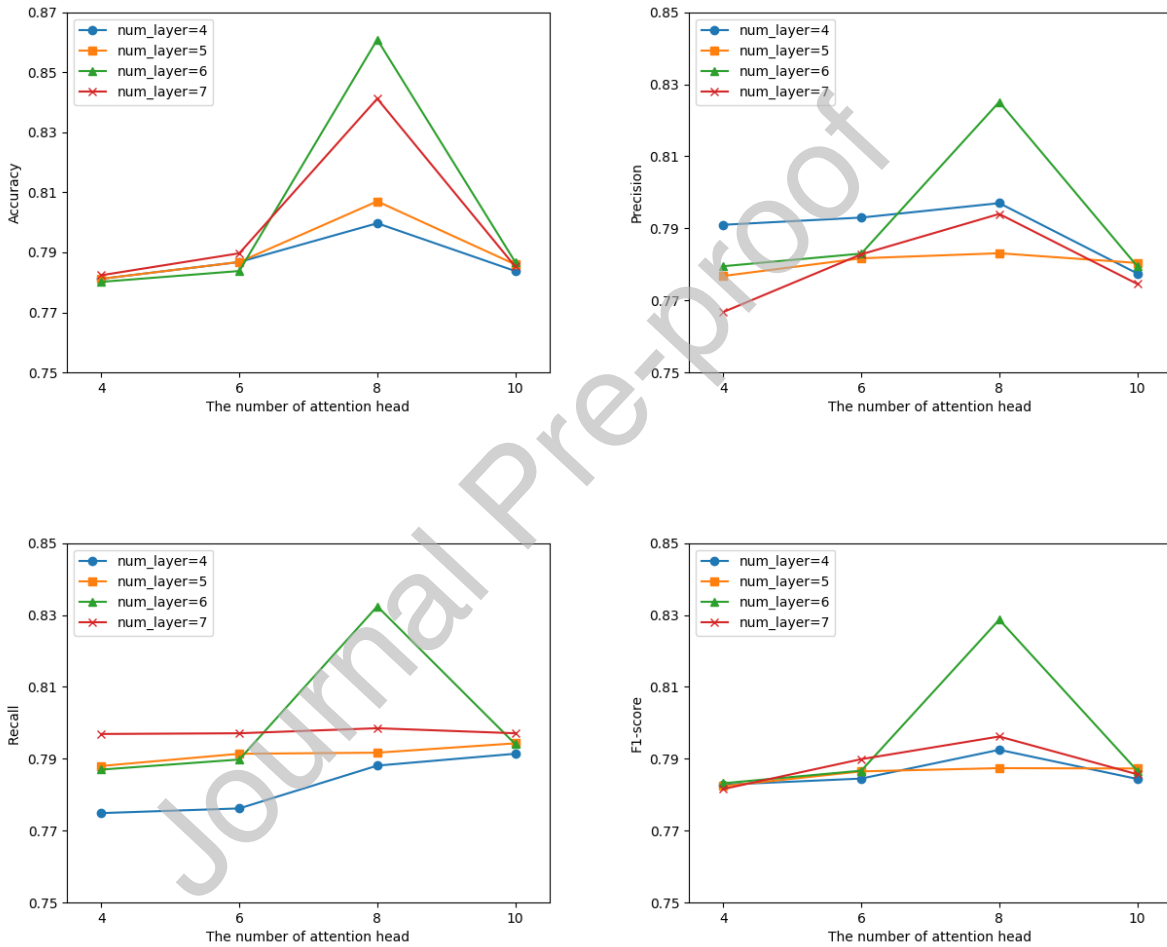


Figure 10: The impact of different attention heads and network layers on final performance

To evaluate the effectiveness of these two parameters Num_heads and num_layers, we initially tested four different network layer configurations, ranging from 4 to 7 layers in increments of 1. For each network layer configuration, we adjusted the number of attention heads from 4 to 10, increasing by increments of 2. Simultaneously, we monitored the impact of varying numbers of network layers and attention heads on experimental metrics, including Accuracy, Precision, Recall, and F1-score.

The impact of network layers and attention heads on prediction performance was evaluated on our datasets, and Figure 10 shows the results of Reveal dataset as examples. The full results for the remaining two datasets, which follow the same evaluation methods and provide consistent findings, are made available in our [GitHub Repository](#) to

ensure transparency and reproducibility without overloading the paper with extensive tables and figures.

By analyzing the line graph in Figure 10, we observe that when the number of network layers remains constant and the number of attention heads is fewer than 8, the performance of the final prediction improves as the number of attention heads increases. The performance reaches its optimal level when the number of attention heads equals 8. For instance, considering the F1-score, when the number of network layers is 6 and the number of attention heads is 4, 6, and 8, the corresponding F1-score values are 78.32%, 78.67%, and 82.87%, respectively. When the number of network layers is reduced to 4, the corresponding F1-score values are 78.28%, 78.45%, and 79.25%, respectively.

This phenomenon can be attributed to the fact that as the number of attention heads increases, the model is able to learn features from multiple perspectives. Different attention heads can focus on capturing various types of dependency relationships, thereby enhancing their understanding and representation of the data.

However, we also observed that when the number of attention heads is set to 10, the performance of the final prediction decreases to varying degrees. For instance, when the number of network layers is 7, the corresponding F1-score values are 78.16%, 78.99%, 79.62%, and 78.57%, respectively. In this case, the performance declines when the number of attention heads is 10. Similarly, when the number of network layers is reduced to 5, the corresponding F1-score values are 78.24%, 78.65%, 78.74%, and 78.73%, respectively, indicating a performance drop when the number of attention heads is also 10. This unusual phenomenon can be attributed to several factors:

1. When the number of network layers remains constant, an increase in the number of heads will significantly raise both computational and storage overhead. More heads require additional storage for parameters and intermediate results, which can hinder the effective training of the model when resources are limited.

2. When the number of attention heads increases beyond a certain threshold, information redundancy may arise among the heads. This can result in the detection of similar patterns during the learning process, causing different heads to fail to provide new, useful information and instead merely reiterate the original data. Such redundancy can ultimately lead to a decline in the model's performance.

On the other hand, we found that when the number of attention heads remains constant, an increase in the number of network layers does not necessarily lead to improved final prediction performance. In fact, it may result in a decline. For instance, when the number of attention heads is set to 6, the F1-scores are 78.45%, 78.65%, 78.67%, and 78.99%, respectively, indicating a consistent upward trend. Conversely, when the number of attention heads is set to 8, the measured values are 79.25%, 78.74%, 82.87%, and 79.62%, respectively, demonstrating irregular fluctuations.

After conducting our analysis, we believe that the fluctuations in performance may be attributed to the following factors:

1. As the number of layers in the Transformer network increases, information be transmitted through more layers. This can lead to a certain degree of information loss, which may limit the enhancement of predictive performance.

2. With the increase in the number of network layers, the number of parameters in the model also rises sharply, which complicates the training process and makes it challenging to achieve optimal performance. Basic features may be supplanted by more complex features in higher layers, and certain network layers may struggle to effectively learn useful features, ultimately impacting the overall performance of the model.

From the experimental results, it is evident that the final prediction performance is better when the number of attention heads is set to 8 and the number of network layers is set to 6. This configuration achieves an optimal balance between prediction performance and training cost. Therefore, we employed these parameter settings in our experiments.

3.6. Threats to the validity

Based on previous experimental results, it can be demonstrated that our method has achieved outstanding performance in the field of vulnerability detection by effectively integrating features from both source codes and binary codes. However, our method still encounters several potential threats to its validity. This section will provide a detailed analysis of both internal and external threats.

3.6.1. Internal validity threats

The internal threats to effectiveness mainly come from the parameter settings during the experimental process. Our method employs pre-trained large models for feature extraction. Given the extensive number of parameters in the model, there are numerous combinations that can significantly affect the final prediction accuracy. We strive to

comprehensively evaluate different parameters and select the optimal configuration within the testing range.

When comparing baseline models, the original papers typically do not provide a complete file that can be directly copied and executed. We strive to reconstruct the model architecture as accurately as possible based on the descriptions in these papers. In our restoration process, we adhere closely to the illustrations provided in the articles. For parameter configurations that are not explicitly stated, we make every effort to optimize performance to ensure a fair comparison.

3.6.2. External validity threats

The primary external threat to validity stems from the limited generalization ability of the current datasets. The most notable limitation of our approach is its dependence on binary codes, yet not every code snippet can be compiled directly. Although ChatGPT can alleviate this issue through automatic completion, multiple revisions are often required, and some fragments ultimately remain uncompileable. Since this process incurs substantial additional costs, we restricted our experiments to only three datasets. Nevertheless, these datasets are representative and widely adopted in vulnerability-detection research, thus, we believe the experimental results are still persuasive. Moreover, for software developers, obtaining the binary codes of a product is generally straightforward, indicating that our method retains practical value.

To verify whether the extra information introduced by the completed code influences the results, we resorted to baselines for indirect validation because the corresponding binary files for the code snippets were unavailable. In subsequent work, however, we plan to expand the experimental scope and seek more direct evidence to enhance the comprehensiveness and persuasiveness of our conclusions.

Finally, our experimental samples are based on C, C++, and Java, the performance of our method on other programming languages remains to be validated. Nonetheless, it should be noted that, regardless of the programming language, source codes can be subjected to similar analysis via pre-trained models, and on the same target machine the internal execution logic (binary codes) of a given function remains comparable. Consequently, our approach is likely applicable to other languages as well. We will further evaluate its generalization ability through more extensive datasets in future work.

4. Related work

At present, software vulnerability detection can be regarded as a hot topic in Software Engineering. In order to minimize the incidence of software vulnerability, researchers have proposed various vulnerability detection methods using different technologies from multiple perspectives.

A kind of traditional but effective methods are based on static code analysis according to pre-defined rules and patterns, such as popular Fortify[40], FlawFinder[41] and Checkmax[42]. They define specific function call patterns or other code structures, and inspect whether the codes can match these patterns to identify their potential risk. However, since most of rules are manually defined, their effectiveness is often limited to certain kinds of defects.

Machine learning becomes popular[43, 44, 45, 46], so some vulnerability detection methods also start to summarize features from source codes or information related to source codes for model training. Common features often include static code analysis (such as complexity metrics, code lines), dynamic behavior monitoring (such as execution path coverage), developer information (such as submission frequency, experience level), and many other aspects. [47] is a software defect prediction method based on multi-source uncertain information fusion and multi task multi view learning (MTMV). This method extracts multi view features from multi-source static analysis results, including code metric features (CMF), defect spatial structure features (SSF), and defect typical semantic features (TSF), to reflect uncertain defect location distribution and semantic information. Then, utilizing the global attention mechanism and the Convolutional Gated Recurrent Unit (CGRU) encoder in the MTMV model, the uncertain vulnerability information in multiple views is fully integrated, and effective multi-objective software vulnerability detection is achieved. GHFNet[48] is a novel neural network model used for software vulnerability detection. This model combines high-level semantic features (HSF) and weighted static features (WSF) for multi-level feature extraction, and adaptively assigns weights to these two features using homomorphic gated fusion units (GHF) to form fused features for code vulnerability detection.

Moreover, the deep learning based methods automatically extract the required feature information from the dataset by defining models[24, 49, 11, 50]. Most of these deep learning methods are based on Abstract Syntax Trees

(AST)[51], Program Dependency Graphs (PDG), Control Flow Graphs (CFG), and Call Graphs (CPG)[52], among others. For instance, Acos[32] uses the word2vec model to encode the source codes, and then use cosine similarity to quantify the similarity between the source codes vector and the vulnerability keyword vector, in order to determine whether there is a vulnerability. AMPLE[31] is a vulnerability detection model that combines graph simplification and enhanced graph representation learning. This model simplifies the graph structure through types and variables, reduces node distances, and uses edge aware graph convolutional networks (EA-GCN) and kernel scale representation modules (KSR) to capture heterogeneous edge information and distant node relationships in the graph, significantly improving vulnerability detection performance.

The above three kinds of methods mostly extract a single type of feature and input it into the model for prediction. Software features have multimodal characteristics, and integrating multiple types of features can provide a more comprehensive representation to more accurately depict and analyze software behavior.

Recently, various information fusion methods[53, 54] for prediction have become mainstream, which combine information from different dimensions by selecting them, Concoction[55] is a software vulnerability prediction system that combines static codes information and dynamic symbol execution tracing. The system utilizes deep learning techniques to improve the accuracy of vulnerability detection by combining the static features of program data and control flow graph (PDCG) with the dynamic features of carefully selected symbols on the execution path. [56] combines the multimodal data of the Source Code Layer (SC), the Build Based Layer (BB) and the EVM Bytecode Layer (EVMB) of the Ethereum virtual machine, and using static analysis technology, we can extract the single modal settings and inter modal settings embedded in code and graph, Advanced AI models such as bidirectional long short-term memory network (bi LSTM), text convolutional neural network (textCNN), and random forest (RF) using self-attention mechanism are used to train and make decisions on features to extract joint multimodal feature representations.

Although existing multimodal methods have certain advantages, they still have certain shortcomings: 1) These methods usually rely on the quality of predefined features, and once the predefined features are inaccurate, they will have a significant impact on the final results. 2) The objects used in almost all of them are source codes, while the execution information is often ignored. 3) Fusion methods in them also pay less attention to the correlation between features.

Differently, our approach aims at obtaining the representation of software without pre-defined features, so larger pre-trained models are used in this process to gain high-quality features automatically. Furthermore, binary codes are introduced into the prediction model so that more detailed execution information could be used to improve the performance. Finally, we design the fusion strategy to better utilize the information from different modalities according to their traits, so that fusion process can focus on the useful features and filter the redundant one.

5. Conclusion and future work

In this paper, we propose a software vulnerability detection method based on multimodal information fusion from source and binary codes. Specifically, source and binary codes are firstly analyzed by pre-trained models, Unixcoder and ViT, separately to gain their representations. Then, we design a novel fusion strategy that taking the information from source codes as the main body to fuse the supplement information from features of binary codes, so that the software could be comprehensively described. Finally, the fusion vector is used in the classifier to get the vulnerability detection results. To verify the effectiveness of our method, we conduct experiments on three representative datasets: Reveal, Devign, and code_vulnerability_java. The results show that our approach achieves state-of-the-art performance on each: on Reveal it attains 86.09% Accuracy and 82.87 % F1-score; on Devign 84.58% Accuracy and 84.62% F1-score; and on code_vulnerability_java 80.43% Accuracy and 79.58% F1-score. Moreover, the experiments indicate the usefulness of information from binary codes on vulnerability detection, and our fusion strategy can effectively utilize such information while filter the redundant one. This is beneficial for the performance improvement.

In the future, we plan to explore better method to analyze binary codes so that they can play a more important role in the prediction. Moreover, we also want to evaluate our approach on larger datasets to further inspect its practicability. To achieve this goal, corresponding tools are to be developed.

Declaration of interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

The work is funded by Jilin Provincial Science and Technology Development Plan Project No.20240302084GX.

References

- [1] Szymon Stradowski and Lech Madeyski. Costs and benefits of machine learning software defect prediction: Industrial case study. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, FSE 2024, page 92–103, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400706585.
- [2] Jian Zhang, Chong Wang, Anran Li, Wenhan Wang, Tianlin Li, and Yang Liu. Vuladvisor: Natural language suggestion generation for software vulnerability repair. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ASE '24, page 1932–1944, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400712487. doi: 10.1145/3691620.3695555. URL <https://doi.org/10.1145/3691620.3695555>.
- [3] Liyan Song, Leandro Lei Minku, Cong Teng, and Xin Yao. A practical human labeling method for online just-in-time software defect prediction. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2023, page 605–617, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400703270. doi: 10.1145/3611643.3616307. URL <https://doi.org/10.1145/3611643.3616307>.
- [4] Emanuele Iannone, Giulia Sellitto, Emanuele Iaccarino, Filomena Ferrucci, Andrea De Lucia, and Fabio Palomba. Early and realistic exploitability prediction of just-disclosed software vulnerabilities: How reliable can it be? *ACM Trans. Softw. Eng. Methodol.*, 33(6), June 2024. ISSN 1049-331X. doi: 10.1145/3654443. URL <https://doi.org/10.1145/3654443>.
- [5] Mayur Jagtap, Praveen Katragadda, and Pooja Satelkar. Software reliability: Development of software defect prediction models using advanced techniques. In *2022 Annual Reliability and Maintainability Symposium (RAMS)*, pages 1–7, 2022. doi: 10.1109/RAMS51457.2022.9893986.
- [6] Szymon Stradowski and Lech Madeyski. Bridging the gap between academia and industry in machine learning software defect prediction: Thirteen considerations. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, ASE '23, page 1098–1110. IEEE Press, 2024. ISBN 9798350329964. doi: 10.1109/ASE56229.2023.00026. URL <https://doi.org/10.1109/ASE56229.2023.00026>.
- [7] Umamaheswara Sharma Bhutamapuram. Some investigations of machine learning models for software defects. In *Proceedings of the 45th International Conference on Software Engineering: Companion Proceedings*, ICSE '23, page 259–263. IEEE Press, 2023. ISBN 9798350322637. doi: 10.1109/ICSE-Companion58688.2023.00070. URL <https://doi.org/10.1109/ICSE-Companion58688.2023.00070>.
- [8] Chao Ni, Xinrong Guo, Yan Zhu, Xiaodan Xu, and Xiaohu Yang. Function-level vulnerability detection through fusing multi-modal knowledge. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, ASE '23, page 1911–1918. IEEE Press, 2024. ISBN 9798350329964. doi: 10.1109/ASE56229.2023.00084. URL <https://doi.org/10.1109/ASE56229.2023.00084>.
- [9] Yi Li, Shaohua Wang, and Tien N. Nguyen. Vulnerability detection with fine-grained interpretations. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2021, page 292–303, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450385626. doi: 10.1145/3468264.3468597. URL <https://doi.org/10.1145/3468264.3468597>.
- [10] Ruitong Liu, Yanbin Wang, Haitao Xu, Jianguo Sun, Fan Zhang, Peiyue Li, and Zhenhao Guo. Vul-Imgnns: Fusing language models and online-distilled graph neural networks for code vulnerability detection. *Information Fusion*, 115:102748, 2025. ISSN 1566-2535. doi: <https://doi.org/10.1016/j.inffus.2024.102748>. URL <https://www.sciencedirect.com/science/article/pii/S1566253524005268>.
- [11] Bin Yuan, Yifan Lu, Yilin Fang, Yueming Wu, Deqing Zou, Zhen Li, Zhi Li, and Hai Jin. Enhancing deep learning-based vulnerability detection by building behavior graph model. In *Proceedings of the 45th International Conference on Software Engineering*, ICSE '23, page 2262–2274. IEEE Press, 2023. ISBN 9781665457019. doi: 10.1109/ICSE48619.2023.00190. URL <https://doi.org/10.1109/ICSE48619.2023.00190>.
- [12] Sicong Cao, Xiaobing Sun, Xiaoxue Wu, David Lo, Lili Bo, Bin Li, Xiaolei Liu, Xingwei Lin, and Wei Liu. Snopy: Bridging sample denoising with causal graph learning for effective vulnerability detection. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ASE '24, page 606–618, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400712487. doi: 10.1145/3691620.3695057. URL <https://doi.org/10.1145/3691620.3695057>.
- [13] Xun Li, Lei Liu, Yuzhou Liu, and Huaxiao Liu. Detecting android malware: A multimodal fusion method with fine-grained feature. *Information Fusion*, 114:102662, 2025. ISSN 1566-2535. doi: <https://doi.org/10.1016/j.inffus.2024.102662>. URL <https://www.sciencedirect.com/science/article/pii/S1566253524004408>.
- [14] Hoai-Chau Tran, Anh-Duy Tran, and Kim-Hung Le. Detectvul: A statement-level code vulnerability detection for python. *Future Generation Computer Systems*, 163:107504, 2025. ISSN 0167-739X. doi: <https://doi.org/10.1016/j.future.2024.107504>. URL <https://www.sciencedirect.com/science/article/pii/S0167739X24004680>.
- [15] Chatgpt. <https://openai.com/>.
- [16] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. Unixcoder: Unified cross-modal pre-training for code representation. *arXiv preprint arXiv:2203.03850*, 2022.

- [17] Alexey Dosovitskiy. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- [18] Masateru Tsunoda, Akito Monden, Koji Toda, Amjed Tahir, Kwabena Ebo Bennin, Keitaro Nakasai, Masataka Nagura, and Kenichi Matsumoto. Using bandit algorithms for selecting feature reduction techniques in software defect prediction. In *Proceedings of the 19th International Conference on Mining Software Repositories*, MSR '22, page 670–681, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450393034. doi: 10.1145/3524842.3529093. URL <https://doi.org/10.1145/3524842.3529093>.
- [19] Shahid Hussain and Naseem Ibrahim. Empirical investigation of role of meta-learning approaches for the improvement of software development process via software fault prediction. In *Proceedings of the 26th International Conference on Evaluation and Assessment in Software Engineering*, EASE '22, page 413–420, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450396134. doi: 10.1145/3530019.3531333. URL <https://doi.org/10.1145/3530019.3531333>.
- [20] Xin-Cheng Wen, Xincheng Wang, Cuiyun Gao, Shaohua Wang, Yang Liu, and Zhaoquan Gu. When less is enough: Positive and unlabeled learning model for vulnerability detection. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, ASE '23, page 345–357. IEEE Press, 2024. ISBN 9798350329964. doi: 10.1109/ASE56229.2023.00144. URL <https://doi.org/10.1109/ASE56229.2023.00144>.
- [21] Jingzheng Wu, Xiang Ling, Xu Duan, Tianyue Luo, and Mutian Yang. Vuldl: Tree-based and graph-based neural networks for vulnerability detection and localization. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, EASE '24, page 323–332, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400717017. doi: 10.1145/3661167.3661211. URL <https://doi.org/10.1145/3661167.3661211>.
- [22] Anson Pinhero, Anupama M L, Vinod P. C.A. Visaggio, Aneesh N, Abhijith S, and AnanthaKrishnan S. Malware detection employed by visualization and deep neural network. *Computers Security*, 105:102247, 2021. ISSN 0167-4048. doi: <https://doi.org/10.1016/j.cose.2021.102247>. URL <https://www.sciencedirect.com/science/article/pii/S0167404821000717>.
- [23] L. Nataraj, S. Karthikeyan, G. Jacob, and B. S. Manjunath. Malware images: visualization and automatic classification. In *Proceedings of the 8th International Symposium on Visualization for Cyber Security*, VizSec '11, New York, NY, USA, 2011. Association for Computing Machinery. ISBN 9781450306799. doi: 10.1145/2016904.2016908. URL <https://doi.org/10.1145/2016904.2016908>.
- [24] Bozhi Wu, Shangqing Liu, Yang Xiao, Zhiming Li, Jun Sun, and Shang-Wei Lin. Learning program semantics for vulnerability detection via vulnerability-specific inter-procedural slicing. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2023, page 1371–1383, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400703270. doi: 10.1145/3611643.3616351. URL <https://doi.org/10.1145/3611643.3616351>.
- [25] Yizhou Chen, Zeyu Sun, Zhihao Gong, and Dan Hao. Improving smart contract security with contrastive learning-based vulnerability detection. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ICSE '24, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400702174. doi: 10.1145/3597503.3639173. URL <https://doi.org/10.1145/3597503.3639173>.
- [26] Cláudia Mamede, Eduard Pinconschi, and Rui Abreu. A transformer-based ide plugin for vulnerability detection. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, ASE '22, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9781450394758. doi: 10.1145/3551349.3559534. URL <https://doi.org/10.1145/3551349.3559534>.
- [27] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. Deep learning based vulnerability detection: Are we there yet? *IEEE Transactions on Software Engineering*, 48(9):3280–3296, 2022. doi: 10.1109/TSE.2021.3087402.
- [28] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. *Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks*. Curran Associates Inc., Red Hook, NY, USA, 2019.
- [29] Dataset. https://huggingface.co/datasets/maddyruos/code_vulnerability_java.
- [30] Jinyin Chen, Keke Hu, Yue Yu, Zhuangzhi Chen, Qi Xuan, Yi Liu, and Vladimir Filkov. Software visualization and deep transfer learning for effective software defect prediction. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ICSE '20, page 578–589, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450371216. doi: 10.1145/3377811.3380389. URL <https://doi.org/10.1145/3377811.3380389>.
- [31] Xin-Cheng Wen, Yupan Chen, Cuiyun Gao, Hongyu Zhang, Jie M. Zhang, and Qing Liao. Vulnerability detection with graph simplification and enhanced graph representation learning. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 2275–2286, 2023. doi: 10.1109/ICSE48619.2023.00191.
- [32] M Maruf Öztürk. A cosine similarity-based labeling technique for vulnerability type detection using source codes. *Computers & Security*, 146:104059, 2024.
- [33] Zhenhao Luo, Pengfei Wang, Baosheng Wang, Yong Tang, Wei Xie, Xu Zhou, Danjun Liu, and Kai Lu. Vulhawk: Cross-architecture vulnerability detection with entropy-based binary code search. *Proceedings 2023 Network and Distributed System Security Symposium*, 2023. URL <https://api.semanticscholar.org/CorpusID:257501992>.
- [34] Wenxin Tao, Xiaohong Su, Jiayuan Wan, Hongwei Wei, and Weining Zheng. Vulnerability detection through cross-modal feature enhancement and fusion. *Computers & Security*, 132:103341, 2023. ISSN 0167-4048. doi: <https://doi.org/10.1016/j.cose.2023.103341>. URL <https://www.sciencedirect.com/science/article/pii/S0167404823002511>.
- [35] Wenxin Tao, Xiaohong Su, Yekun Ke, Yi Han, Yu Zheng, and Hongwei Wei. Transformer-based statement level vulnerability detection by cross-modal fine-grained features capture. *Knowledge-Based Systems*, 316:113341, 2025. ISSN 0950-7051. doi: <https://doi.org/10.1016/j.knosys.2025.113341>. URL <https://www.sciencedirect.com/science/article/pii/S0950705125003880>.
- [36] Wojciech Zaremba. Recurrent neural network regularization. *arXiv preprint arXiv:1409.2329*, 2014.
- [37] Zhou Zhao, Ashok Srivastava, Lu Peng, and Qing Chen. Long short-term memory network design for analog computing. *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, 15(1):1–27, 2019.
- [38] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In Alessandro Moschitti, Bo Pang, and Walter Daelemans, editors, *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, Doha, Qatar, October 2014. Association for Computational Linguistics. doi: 10.3115/v1/D14-1179.
- [39] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*,

- 2016.
- [40] Fortify. <https://www.microfocus.com/products/fortify/>.
 - [41] Flawfinder. <https://www.dwheeler.com/flawfinder/>.
 - [42] Checkmax. <https://www.checkmarx.com/>.
 - [43] Amad Rizwan Ali, Attique Ur Rehman, Ali Nawaz, Tahir Muhammad Ali, and Muhammad Abbas. An ensemble model for software defect prediction. In *2022 2nd International Conference on Digital Futures and Transformative Technologies (ICoDT2)*, pages 1–5, 2022. doi: 10.1109/ICoDT255437.2022.9787439.
 - [44] Debasish Pradhan and Debendra Muduli. Software defect prediction model using adaboost based random forest technique. In *2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pages 1–6, 2023. doi: 10.1109/ICCCNT56998.2023.10308208.
 - [45] Xiao Yu, Jiqing Rao, Lei Liu, Guancheng Lin, Wenhua Hu, Jacky Wai Keung, Junwei Zhou, and Jianwen Xiang. Improving effort-aware defect prediction by directly learning to rank software modules. *Inf. Softw. Technol.*, 165(C), January 2024. ISSN 0950-5849. doi: 10.1016/j.infsof.2023.107250. URL <https://doi.org/10.1016/j.infsof.2023.107250>.
 - [46] Nikhil Khanna, Ojas Agarwal, and Rahul. Software change prediction using ensemble learning on object oriented metrics. In *2022 6th International Conference on Intelligent Computing and Control Systems (ICICCS)*, pages 1369–1373, 2022. doi: 10.1109/ICICCS53718.2022.9788196.
 - [47] Minghao Yang, Shunkun Yang, and W. Eric Wong. Multi-objective software defect prediction via multi-source uncertain information fusion and multi-task multi-view learning. *IEEE Transactions on Software Engineering*, 50(8):2054–2076, 2024. doi: 10.1109/TSE.2024.3421591.
 - [48] Dingbang Fang, Shaoying Liu, and Ai Liu. Gated homogeneous fusion networks with jointed feature extraction for defect prediction. *IEEE Transactions on Reliability*, 71(2):512–526, 2022. doi: 10.1109/TR.2022.3165115.
 - [49] Cheng Weng, Yihao Qin, Bo Lin, Pei Liu, and Liqian Chen. Matsvd: Boosting statement-level vulnerability detection via dependency-based attention. In *Proceedings of the 15th Asia-Pacific Symposium on Internetware, Internetware '24*, page 115–124, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400707056. doi: 10.1145/3671016.3674807. URL <https://doi.org/10.1145/3671016.3674807>.
 - [50] Md Mahbubur Rahman, Ira Ceka, Chengzhi Mao, Saikat Chakraborty, Baishakhi Ray, and Wei Le. Towards causal deep learning for vulnerability detection. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24*, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400702174. doi: 10.1145/3597503.3639170. URL <https://doi.org/10.1145/3597503.3639170>.
 - [51] Chao Ni, Xin Yin, Kaiwen Yang, Dehai Zhao, Zhenchang Xing, and Xin Xia. Distinguishing look-alike innocent and vulnerable code by subtle semantic representation learning and explanation. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023*, page 1611–1622, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400703270. doi: 10.1145/3611643.3616358. URL <https://doi.org/10.1145/3611643.3616358>.
 - [52] Tiehua Zhang, Rui Xu, Jianping Zhang, Yuze Liu, Xin Chen, Jun Yin, and Xi Zheng. Dshgt: Dual-supervisors heterogeneous graph transformer—a pioneer study of using heterogeneous graph learning for detecting software vulnerabilities. *ACM Trans. Softw. Eng. Methodol.*, 33(8), November 2024. ISSN 1049-331X. doi: 10.1145/3674729. URL <https://doi.org/10.1145/3674729>.
 - [53] Yan Wang, Peng Jia, Xi Peng, Cheng Huang, and Jiayong Liu. Binvuldet: Detecting vulnerability in binary program via decompiled pseudo code and bilstm-attention. *Computers & Security*, 125:103023, 2023.
 - [54] Qianchong Zhao, Cheng Huang, and Liuhu Dai. Vuldeff: Vulnerability detection method based on function fingerprints and code differences. *Knowledge-Based Systems*, 260:110139, 2023. ISSN 0950-7051. doi: <https://doi.org/10.1016/j.knosys.2022.110139>. URL <https://www.sciencedirect.com/science/article/pii/S0950705122012357>.
 - [55] Huanting Wang, Zhanyong Tang, Shin Hwei Tan, Jie Wang, Yuzhe Liu, Hejun Fang, Chunwei Xia, and Zheng Wang. Combining structured static code information and dynamic symbolic traces for software vulnerability prediction. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24*, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400702174. doi: 10.1145/3597503.3639212. URL <https://doi.org/10.1145/3597503.3639212>.
 - [56] Wanqing Jie, Qi Chen, Jiaqi Wang, Arthur Sandor Voundi Koe, Jin Li, Pengfei Huang, Yaqi Wu, and Yin Wang. A novel extended multimodal ai framework towards vulnerability detection in smart contracts. *Information Sciences*, 636:118907, 2023. ISSN 0020-0255. doi: <https://doi.org/10.1016/j.ins.2023.03.132>. URL <https://www.sciencedirect.com/science/article/pii/S0020025523004565>.